

Scholarship & Scheme RAG Engine

Phase 1 Production Documentation: Ingestion, Metadata Extraction & Structure-Aware Chunking

Core Milestone: Phase 1 transforms raw, unstructured, multi-column government circular PDFs into high-precision, table-intact Markdown text chunks accompanied by structured JSON metadata sidecars.

Everyday Analogy: Imagine receiving hundreds of messy paper files with grids, stamps, and fine print. Phase 1 is like a smart digital scanner and librarian that reads every document, extracts the key rules onto an index card (metadata), types out clean, neat text pages (Markdown), and clips related paragraphs and tables together cleanly into structured folders (chunks).

1. Phase 1 Objectives & Key Achievements

The primary goal of Phase 1 is to establish a zero-loss data ingestion pipeline. Standard PDF tools scramble multi-column layouts and break tables into floating numbers. Phase 1 fixes this by converting PDFs into structured Markdown, retaining 100% of table geometries and header hierarchies.

- Layout-Aware PDF Conversion:** Processed complex PDF circulars into Markdown without losing column or table structure.
- Automated Metadata Extraction:** Extracted structured JSON sidecars (State, Eligible Categories, Income Caps) for fast pre-filtering in Phase 2.
- Header & Table-Intact Chunking:** Split documents logically by headers (#, ##) while ensuring Markdown tables (|) stay complete in single chunks.
- Simultaneous Admin Dashboard:** Built a 3-tab Streamlit interface (Upload, Metadata Inspector, Chunk Quality Auditor) for live testing.
- Enterprise GitHub Setup:** Organized a production layout (src/ingestion/, data/processed/) with error logging and clean version control.

2. Tools & Frameworks Used (Simple Purpose Map)

Tool / Framework	Role in Pipeline	Everyday Analogy / Simple Purpose
Python 3.12	Core Programming Language	The engine that runs all scripts and automation pipeline logic.
Docling (by IBM)	Layout & Vision PDF Converter	A digital eye that looks at page layout, recognizing tables & columns.
Regex & JSON	Rule-Based Metadata Extractor	A smart highlighter that extracts income caps & categories into index cards.
Markdown Header Chunker	Structure-Preserving Splitter	Smart scissors that cut text into readable sections without splitting tables.
Streamlit	Admin Dashboard & UI	The car dashboard allowing visual testing, document upload, and chunk auditing.
Git & GitHub	Code Vault & Version Control	Google Drive for code that tracks updates and hosts the repository online.

3. Connection: Phase 1 Output vs Phase 2 Input

Phase 1 prepares the raw data so Phase 2 can search it at lightning speed. Here is how they connect:

Phase Aspect	Phase 1: Ingestion & Data Preparation (Completed)	Phase 2: Hybrid Retrieval & Vector Store (Next Step)
Input Data	Raw Unstructured PDF Circulars & Notifications	Clean Markdown Chunks (all_chunks.json) & Metadata (_meta.json)
Core Process	PDF Conversion + Table Preservation + Metadata Tagging	Vector Embedding Generation (BAAI/bge-m3) + FAISS & BM25 Indexing
Output Artifact	Structured Chunks & Metadata Sidecars	Searchable Vector Database + Hybrid Keyword Index
System Analogy	Building and organizing the books on the library shelves	Creating the super-fast digital library catalog for instant search

4. How to Explain Phase 1 in an Interview

30-Second Elevator Pitch:
'In Phase 1 of our Scholarship RAG project, we built a production-scale PDF ingestion pipeline. Standard PDF extractors scramble multi-column government circulars and break income/caste eligibility tables. We used Docling to parse PDFs into clean Markdown while keeping tables intact. We also built an automated regex rule engine to extract metadata sidecars (annual income caps, caste categories, state) for pre-filtering, and split documents using structure-aware header chunking into all_chunks.json. We validated the entire pipeline using a custom 3-tab Streamlit admin dashboard.'